Python and Programming 1

Programming basics and algorithms

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

Today's outline

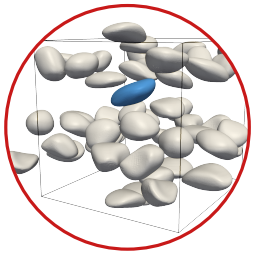
- Introduction
- Refresher
- New topics in programming
- Modules
- Conclusions
- Exercises

Today's outline

- Introduction
- Refresher
- New topics in programming
- Modules
- Conclusions
- Exercises

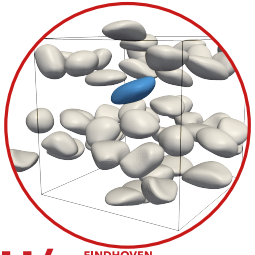
Why should you learn something about programming?

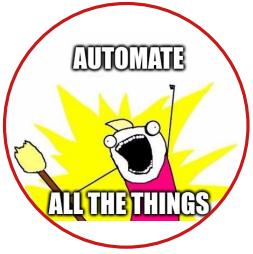
- Scientific analyses depend more than ever on computer programs and simulation methods



Why should you learn something about programming?

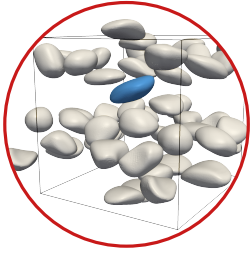
- Scientific analyses depend more than ever on computer programs and simulation methods
- Knowledge of programming allows you to automate routine tasks

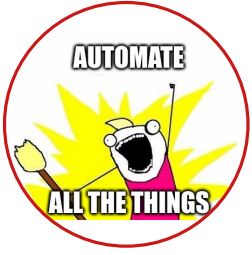




Why should you learn something about programming?

- Scientific analyses depend more than ever on computer programs and simulation methods
- Knowledge of programming allows you to automate routine tasks
- Ability to understand algorithms by inspection of the code

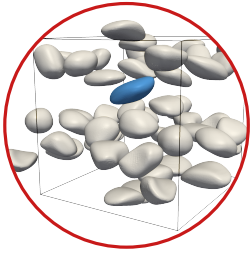


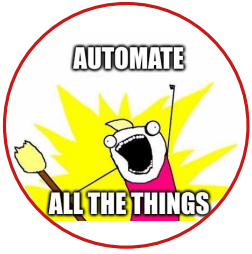




Why should you learn something about programming?

- Scientific analyses depend more than ever on computer programs and simulation methods
- Knowledge of programming allows you to automate routine tasks
- Ability to understand algorithms by inspection of the code
- Learn to think by dissecting a problem into smaller, easier to solve, parts









Introduction to programming

What is a program?

A program is a sequence of instructions that is written to perform a certain task on a computer.

- The computation might be something mathematical, a symbolic operation, image analysis, etc.

Data → Algorithm → Result

A program typically consists of three parts:

1. Input (Get the radius of a circle)
2. Operations (Compute and store the area of the circle)
3. Output (Print the area to the screen)

Python help, documentation, resources

- Use the course materials provided for 6BBR06 - Programming and Linear Algebra
- Refer to the Python documentation at Official documentation.
  - Try for instance: `help(print)` or `help(help)`.
- Other packages that we will use:
  - NumPy documentation
  - Matplotlib documentation
  - SciPy documentation
- Python Crash Course, 3rd Edition by Eric Matthes
- A Whirlwind Tour of Python by Jake Vanderplas
- Introduction to Scientific Programming with Python by Joakim Sundnes
- Python Programming And Numerical Methods: A Guide For Engineers And Scientists by Kong, Siauw and Bayen
- Search the web, Reddit, YouTube, etc.

About your editor

- There are several keyboard shortcuts (vary with editor) that will make coding much more efficient. For VSCode:

About your editor

- There are several keyboard shortcuts (vary with editor) that will make coding much more efficient. For VSCode:
  - Typing part of a command and pressing Tab completes the command and lists the possibilities

About your editor

- There are several keyboard shortcuts (vary with editor) that will make coding much more efficient. For VSCode:
  - Typing part of a command and pressing Tab completes the command and lists the possibilities
  - Move lines or selections up and down with Alt + Up / Down

About your editor

- There are several keyboard shortcuts (vary with editor) that will make coding much more efficient. For VSCode:
  - Typing part of a command and pressing Tab completes the command and lists the possibilities
  - Move lines or selections up and down with Alt + Up / Down
  - Duplicate lines with Shift + Alt + Down

About your editor

- There are several keyboard shortcuts (vary with editor) that will make coding much more efficient. For VSCode:
  - Typing part of a command and pressing Tab completes the command and lists the possibilities
  - Move lines or selections up and down with Alt + Up / Down
  - Duplicate lines with Shift + Alt + Down
  - You can bind keystrokes such as Ctrl + F5 to run the current script

About your editor

- There are several keyboard shortcuts (vary with editor) that will make coding much more efficient. For VSCode:
  - Typing part of a command and pressing Tab completes the command and lists the possibilities
  - Move lines or selections up and down with Alt + Up / Down
  - Duplicate lines with Shift + Alt + Down
  - You can bind keystrokes such as Ctrl + F5 to run the current script
  - Comment/uncomment lines with Ctrl + /

About your editor

- There are several keyboard shortcuts (vary with editor) that will make coding much more efficient. For VSCode:
  - Typing part of a command and pressing Tab completes the command and lists the possibilities
  - Move lines or selections up and down with Alt + Up / Down
  - Duplicate lines with Shift + Alt + Down
  - You can bind keystrokes such as Ctrl + F5 to run the current script
  - Comment/uncomment lines with Ctrl + /
  - Set up snippets for often used code blocks (e.g. importing packages)

About your editor

- There are several keyboard shortcuts (vary with editor) that will make coding much more efficient. For VSCode:
  - Typing part of a command and pressing Tab completes the command and lists the possibilities
  - Move lines or selections up and down with Alt + Up / Down
  - Duplicate lines with Shift + Alt + Down
  - You can bind keystrokes such as Ctrl + F5 to run the current script
  - Comment/uncomment lines with Ctrl + /
  - Set up snippets for often used code blocks (e.g. importing packages)
- If a computation takes too long, you can press Ctrl + C to stop the program and return to the command line. Stored variables may contain incomplete results.

About your editor

- There are several keyboard shortcuts (vary with editor) that will make coding much more efficient. For VSCode:
  - Typing part of a command and pressing Tab completes the command and lists the possibilities
  - Move lines or selections up and down with Alt + Up / Down
  - Duplicate lines with Shift + Alt + Down
  - You can bind keystrokes such as Ctrl + F5 to run the current script
  - Comment/uncomment lines with Ctrl + /
  - Set up snippets for often used code blocks (e.g. importing packages)
- If a computation takes too long, you can press Ctrl + C to stop the program and return to the command line. Stored variables may contain incomplete results.
- Python files must be in the *current working directory* or in the Python *path*, the locations where Python searches for a command. If you try to run a script that is not in the path, Python will throw an Exception/Error.

Today's outline

- Introduction
- Refresher
- New topics in programming
- Modules
- Conclusions
- Exercises

Terminology

Variable Piece of data stored in the computer memory, to be referenced and/or manipulated

Function Piece of code that performs a certain operation/sequence of operations on given input

Operators Mathematical operators (e.g. `+` `-` `*` or `/`), relational (e.g. `<` `>`or `==`, and logical operators (**`and`**, **`or`**)

Script Piece of code that performs a certain sequence of operations without specified input/output

Expression A command that combines variables, functions, operators and/or values to produce a result.

Variables in Python

- Python stores variables in the *namespace*
- You should recognize the difference between the *identifier* of a variable (its name, e.g. `x`, `setpoint_p`), and the data that it actually stores (e.g. 0.5)
- Python also defines a number of functions by default, e.g. `min`, `max` or `sum`.
  - A list of built-in methods is given by `dir(__builtins__)`
- You can assign a variable by the = sign:

```
>>> x = 4*3
>>> x
12
```

- If you don't assign a variable, it will be stored in _
- In most text editors, all variables are cleared automatically before the next execution.

Datatypes and variables

Python uses different types of variables:

Datatype	Example
`str`	`'Wednesday'`
`int`	`15`
`float`	`0.15`
`list`	`[0.0, 0.1, 0.2, 'Hello', ['Another','List']]`
`dict`	`{'name': 'word', "n": 2}`
`bool`	`False`
`tuple`	`(True, False)`

Datatypes and variables

Python uses different types of variables:

Datatype	Example
`str`	`'Wednesday'`
`int`	`15`
`float`	`0.15`
`list`	`[0.0, 0.1, 0.2, 'Hello', ['Another','List']]`
`dict`	`{'name': 'word', "n": 2}`
`bool`	`False`
`tuple`	`(True, False)`

Everything in Python is an object. You can use the `dir()` function to query the possible methods on an object of a datatype (e.g. `(dir(list))`, `dir(28)` or `dir("Yes!")`).

Strings and f-strings

- Strings are sequences of characters, enclosed by either single quotes `'...'` or double quotes `"..."`.
- Methods to manipulate strings include `.upper()`, `.lower()`, `.replace()`, `.split()`, `.index()`, etc.
- Using f-strings are a preferred way to format output (Python 3.6+)
- Readable, compact, explicit
- Formatting affects display, not the numerical value

```
x = 3.1415926535
n = 42
hello_str = "Hello, world!"
value_fstr = f"x is approximate {x:.2f}"

print(f"x = {x}")
print(f"n = {n} and is type {type(n)}")
print(f"{x=}")
print(hello_str)
print(value_fstr)
```

```
x = 3.1415926535
n = 42 and is type <class 'int'>
x=3.1415926535
Hello, world!
x is approximate 3.14
```

Lists (refresher)

- Ordered, mutable containers
- Methods to add, remove, sort, manipulate elements
- Concatenate lists using `+`, or multiply using `*`
- Zero-based indexing, slicing of multiple elements using `:`
- Typical numerical use: storing samples, time series

```
1 my_list_of_numbers = [1, 2, 3]
2 my_list_of_numbers.append(4) # Now: [1, 2, 3, 4]
```

Lists (refresher)

- Ordered, mutable containers
- Methods to add, remove, sort, manipulate elements
- Concatenate lists using `+`, or multiply using `*`
- Zero-based indexing, slicing of multiple elements using `:`
- Typical numerical use: storing samples, time series

```
my_list_of_numbers = [1, 2, 3]
my_list_of_numbers.append(4) # Now: [1, 2, 3, 4]
my_list_of_strings = ["apple", "banana", "cherry"]
my_list_of_strings[1] = "blueberry" # Now: ["apple", "blueberry", "cherry"]
```

Lists (refresher)

- Ordered, mutable containers
- Methods to add, remove, sort, manipulate elements
- Concatenate lists using `+`, or multiply using `*`
- Zero-based indexing, slicing of multiple elements using `:`
- Typical numerical use: storing samples, time series

```
my_list_of_numbers = [1, 2, 3]
my_list_of_numbers.append(4) # Now: [1, 2, 3, 4]
my_list_of_strings = ["apple", "banana", "cherry"]
my_list_of_strings[1] = "blueberry" # Now: ["apple", "blueberry", "cherry"]
my_list_of_booleans = [True] * 4 # [True, True, True, True]
```

Lists (refresher)

- Ordered, mutable containers
- Methods to add, remove, sort, manipulate elements
- Concatenate lists using `+`, or multiply using `*`
- Zero-based indexing, slicing of multiple elements using `:`
- Typical numerical use: storing samples, time series

```
my_list_of_numbers = [1, 2, 3]
my_list_of_numbers.append(4) # Now: [1, 2, 3, 4]
my_list_of_strings = ["apple", "banana", "cherry"]
my_list_of_strings[1] = "blueberry" # Now: ["apple", "blueberry", "cherry"]
my_list_of_booleans = [True] * 4 # [True, True, True, True]
all_numbers = list(range(5)) + [10, 20, 30] # [0, 1, 2, 3, 4, 10, 20, 30]
```

Lists (refresher)

- Ordered, mutable containers
- Methods to add, remove, sort, manipulate elements
- Concatenate lists using `+`, or multiply using `*`
- Zero-based indexing, slicing of multiple elements using `:`
- Typical numerical use: storing samples, time series

```
my_list_of_numbers = [1, 2, 3]
my_list_of_numbers.append(4) # Now: [1, 2, 3, 4]
my_list_of_strings = ["apple", "banana", "cherry"]
my_list_of_strings[1] = "blueberry" # Now: ["apple", "blueberry", "cherry"]
my_list_of_booleans = [True] * 4 # [True, True, True, True]
all_numbers = list(range(5)) + [10, 20, 30] # [0, 1, 2, 3, 4, 10, 20, 30]
my_heterogeneous_list = [1, "apple", 3.14, True, [1, 2], {"a": 1}]
```

Lists (refresher)

- Ordered, mutable containers
- Methods to add, remove, sort, manipulate elements
- Concatenate lists using `+`, or multiply using `*`
- Zero-based indexing, slicing of multiple elements using `:`
- Typical numerical use: storing samples, time series

```
my_list_of_numbers = [1, 2, 3]
my_list_of_numbers.append(4) # Now: [1, 2, 3, 4]
my_list_of_strings = ["apple", "banana", "cherry"]
my_list_of_strings[1] = "blueberry" # Now: ["apple", "blueberry", "cherry"]
my_list_of_booleans = [True] * 4 # [True, True, True, True]
all_numbers = list(range(5)) + [10, 20, 30] # [0, 1, 2, 3, 4, 10, 20, 30]
my_heterogeneous_list = [1, "apple", 3.14, True, [1, 2], {"a": 1}]
```

While lists *can* be heterogeneous, it's often best to keep them homogeneous for easier processing.

Exercise: Lists practice

Given a vector

$$x = [2\ 4\ 6\ 8\ 10\ 12\ 14\ 16\ 18\ 20\ 30\ 40\ 50\ 60\ 70\ 80]$$

- Define the vector using `range`'s, without typing all individual elements

Exercise: Lists practice

Given a vector

$$x = [2\ 4\ 6\ 8\ 10\ 12\ 14\ 16\ 18\ 20\ 30\ 40\ 50\ 60\ 70\ 80]$$

- Define the vector using `range`'s, without typing all individual elements

```
1 x = list(range(2,20,2)) + list(range(20,90,10))
```

Exercise: Lists practice

Given a vector

$$x = [2\ 4\ 6\ 8\ 10\ 12\ 14\ 16\ 18\ 20\ 30\ 40\ 50\ 60\ 70\ 80]$$

- Define the vector using `range`'s, without typing all individual elements

```
x = list(range(2,20,2)) + list(range(20,90,10))
```

- Investigate the meaning of the following commands:

```
x[2]
x[0:5]
x[:-1]
y = x[4:]
y[3]
y.pop(3)
sum(x)
max(x)
min(x)
x[::-1]
```

Dictionaries (refresher)

- Mutable, unordered storage of key–value pairs
- Fast lookup of arbitrary values (i.e. objects)
- Methods include `.keys`, `.values`, `.items`
- Typical use: parameters, metadata

```
1 params = {"dt": 0.01, "T": 10.0, "method": "RK4"}
2 print(f"Time step: {params["dt"]} s") # Access value via key
```

Dictionaries (refresher)

- Mutable, unordered storage of key–value pairs
- Fast lookup of arbitrary values (i.e. objects)
- Methods include `.keys`, `.values`, `.items`
- Typical use: parameters, metadata

```
params = {"dt": 0.01, "T": 10.0, "method": "RK4"}
print(f"Time step: {params["dt"]} s") # Access value via key
params["dt"] = 0.02 # Modify value
params["TotalSteps"] = int(params["T"] / params["dt"]) # Add new key--value pair
print(f"New timestep: {params["dt"]} s, total steps: {params["TotalSteps"]}")
```

Dictionaries (refresher)

- Mutable, unordered storage of key–value pairs
- Fast lookup of arbitrary values (i.e. objects)
- Methods include `.keys`, `.values`, `.items`
- Typical use: parameters, metadata

```
params = {"dt": 0.01, "T": 10.0, "method": "RK4"}
print(f"Time step: {params["dt"]} s") # Access value via key
params["dt"] = 0.02 # Modify value
params["TotalSteps"] = int(params["T"] / params["dt"]) # Add new key--value pair
print(f"New timestep: {params["dt"]} s, total steps: {params["TotalSteps"]}")
```

```
Time step: 0.01 s
New timestep: 0.02 s, total steps: 500
```

Tuples (refresher)

- Ordered, immutable containers
- Methods to count, find elements
- Zero-based indexing, slicing
- Typical numerical use: fixed collections of related values

```
my_tuple = (1, 2, 3)
coordinate_point = (3, 4)
record = ("CO2", 44.01, "g/mol")
```

Tuples (refresher)

- Ordered, immutable containers
- Methods to count, find elements
- Zero-based indexing, slicing
- Typical numerical use: fixed collections of related values

```
my_tuple = (1, 2, 3)
coordinate_point = (3, 4)
record = ("CO2", 44.01, "g/mol")
```

Use tuples to store values that should not change during program execution:

```
print(my_tuple)
my_tuple[1] = 4
```

Tuples (refresher)

- Ordered, immutable containers
- Methods to count, find elements
- Zero-based indexing, slicing
- Typical numerical use: fixed collections of related values

```
my_tuple = (1, 2, 3)
coordinate_point = (3, 4)
record = ("CO2", 44.01, "g/mol")
```

Use tuples to store values that should not change during program execution:

```
print(my_tuple)
my_tuple[1] = 4
```

```
(1, 2, 3)
Traceback (most recent call last):
  File "/home/ivo/Repositories/nm-slides/test_scripts.py", line 5, in <module>
    my_tuple[1] = 4
    ~~~~~~~~^^^
TypeError: 'tuple' object does not support item assignment
```

Loops (refresher)

- `for`: iterate over sequences
- `while`: iterate until condition fails
- `break` to exit loop early, `continue` to skip to next iteration

```
for i in range(5):
    print(f"Iteration {i}")
```

Loops (refresher)

- `for`: iterate over sequences
- `while`: iterate until condition fails
- `break` to exit loop early, `continue` to skip to next iteration

```
for i in range(5):
   print(f"Iteration {i}")
```

```
Index 0, Value 10
Index 1, Value 20
Index 2, Value 30
Index 3, Value 40
```

Loops (refresher)

- Use **`enumerate`** to loop over indices and values
- If you want to loop over two or more lists in parallel, use **`zip`**
- Advanced iteration tools are available in the `itertools` module

```
L = [10, 20, 30, 40]
for i, val in enumerate(L):
    print(f"Index {i}, Value {val}")
```

```
Index 0, Value 10
Index 1, Value 20
Index 2, Value 30
Index 3, Value 40
```

```
M_list = ['a','b','c','d','e']
N_list = [1,2,3,4,5]

for m,n in zip(M_list, N_list):
    print(f"Value {m = }, Value {n = }")
```

```
Value m = 'a', Value n = 1
Value m = 'b', Value n = 2
Value m = 'c', Value n = 3
Value m = 'd', Value n = 4
Value m = 'e', Value n = 5
```

Conditional statements (refresher)

- Control flow based on logic
- Essential for algorithms

```
temp = 10
if temp < 0:
   state = "solid"
elif 0 <= temp < 100:
   state = "liquid"
else:
   state = "gas"

print(f"At {temp} C, water is in {state} state." )
```

Conditional statements (refresher)

- Control flow based on logic
- Essential for algorithms

```
temp = 10
if temp < 0:
   state = "solid"
elif 0 <= temp < 100:
   state = "liquid"
else:
   state = "gas"

print(f"At {temp} C, water is in {state} state." )
```

Check membership:

```
states = ["Good", "Bad", "Ugly"]
if "Good" in states:
   print("We have a good state!")
else:
   print("No good states found.")
```

Ternary operator:

```
score = "Good" if points >= 75 else "Bad"
```

Exercise

- Loop over a list of values (create a list with at least 8 values)
- Count how many are negative
- Stop early if a value exceeds 10
- Print the count of negative values found before stopping

Exercise

- Loop over a list of values (create a list with at least 8 values)
- Count how many are negative
- Stop early if a value exceeds 10
- Print the count of negative values found before stopping

```
values = [3, -1, 4, -2, 5, 11, -3, 6]
neg_count = 0
for v in values:
    if v < 0:
        neg_count += 1
    if v > 10:
        break
print(f"Number of negative values before exceeding 10: {neg_count}")
```

Exercise

- Loop over a list of values (create a list with at least 8 values)
- Count how many are negative
- Stop early if a value exceeds 10
- Print the count of negative values found before stopping

```
values = [3, -1, 4, -2, 5, 11, -3, 6]
neg_count = 0
for v in values:
    if v < 0:
        neg_count += 1
    if v > 10:
        break
print(f"Number of negative values before exceeding 10: {neg_count}")
```

```
Number of negative values before exceeding 10: 2
```

Functions

Functions help in organizing code, improving readability, efficiency, and avoiding repetition.

- Abstraction makes sure you produce fewer bugs
- Parameters give flexibility, default values make sure that functions can be used with minimal input

```
def f(x, a):
    return a*x**2

y = f(2.0, 3.0) # y = 3 * (2.0)^2 = 12.0
```

Functions

Functions help in organizing code, improving readability, efficiency, and avoiding repetition.

- Abstraction makes sure you produce fewer bugs
- Parameters give flexibility, default values make sure that functions can be used with minimal input

```
def f(x, a):
   return a*x**2

y = f(2.0, 3.0) # y = 3 * (2.0)^2 = 12.0
```

Define default parameters to make functions easier to use:

```
def f(x, a=5.0):
   return a*x**2

y = f(2.0) # y = 5 * (2.0)^2 = 20.0
```

Exercise: Functions practice

Define a function that computes the factorial of a number, $n!$:

$$n! = 1 \times 2 \times 3 \times \ldots \times (n-1) \times n$$

Compute $\exp(x)$ using the Taylor series, iterate until the change is smaller than $1 \cdot 10^{-6}$:

$$\exp(x) = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots + \frac{x^n}{n!}$$

Exercise: Functions practice

Define a function that computes the factorial of a number, *n*!:

$$n! = 1 \times 2 \times 3 \times \ldots \times (n-1) \times n$$

```python
def factorial(n):
    """Compute the factorial of n"""
    x = 1
    for i in range(1,n+1):
        x *= i
    return x
```

Compute $\exp(x)$ using the Taylor series, iterate until the change is smaller than $1 \cdot 10^{-6}$:

$$\exp(x) = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots + \frac{x^n}{n!}$$

```python
def exponential(x, eps=1.0e-6):
    """Compute exponential of x with accuracy eps
          (default: 1.0e-6)"""
    i = 0
    taylor_terms = [x**i/factorial(i)]

    while taylor_terms[-1] >= eps:
        i += 1
        taylor_terms.append(x**i/factorial(i))

    return sum(taylor_terms), i
```

Advanced topic: Recursion

- In order to understand recursion, one must first understand recursion

Advanced topic: Recursion

- In order to understand recursion, one must first understand recursion
- A recursive function includes a call to itself (a function within a function)

Advanced topic: Recursion

- In order to understand recursion, one must first understand recursion
- A recursive function includes a call to itself (a function within a function)
  - This could lead to infinite calls;
  - A base case is required so that recursion is stopped;
  - Base case does not call itself, simply returns.



Advanced topic: Recursion

- In order to understand recursion, one must first understand recursion

Advanced topic: Recursion

- In order to understand recursion, one must first understand recursion
- A recursive function includes a call to itself (a function within a function)

Advanced topic: Recursion

- In order to understand recursion, one must first understand recursion
- A recursive function includes a call to itself (a function within a function)
  - This could lead to infinite calls;
  - A base case is required so that recursion is stopped;
  - Base case does not call itself, simply returns.



Recursion: example

```
def mystery(a, b):
  if b == 1:
     # Base case
     return a
  else:
     # Recursive function call
     return a + mystery(a, b-1)
```

Recursion: example

```
def mystery(a, b):
  if b == 1:
     # Base case
     return a
  else:
     # Recursive function call
     return a + mystery(a, b-1)
```

- What does this function do?

Recursion: example

```
def mystery(a, b):
  if b == 1:
     # Base case
     return a
  else:
     # Recursive function call
     return a + mystery(a, b-1)
```

- What does this function do?
- Can you spot the error?

Recursion: example

```
def mystery(a, b):
  if b == 1:
     # Base case
     return a
  else:
     # Recursive function call
     return a + mystery(a, b-1)
```

- What does this function do?
- Can you spot the error?
- How deep can you go? Which values of b don't work anymore?

Practice

Define a function that computes the factorial of a number, $n!$, using recursion:

$$n! = 1 \times 2 \times 3 \times \ldots \times (n-1) \times n$$

Practice

Define a function that computes the factorial of a number, $n!$, using recursion:

$$n! = 1 \times 2 \times 3 \times \ldots \times (n-1) \times n$$

```
def factorial(n):
    """Compute the factorial of n"""
    assert isinstance(n,int) and n >= 0, 'Use positive integers only'

    if n==1:
      return 1
    else:
      return n*factorial(n-1)
```

Lambda functions (1)

Consider the mathematical function $f(x) = x^2 + e^x$ defined as a Python function block:

```
def f(x):
    from math import exp
    return x**2 + exp(x)
```

Lambda functions (1)

Consider the mathematical function $f(x) = x^2 + e^x$ defined as a Python function block:

```
def f(x):
    from math import exp
    return x**2 + exp(x)
```

Lambda functions provide a more compact way of defining simple functions, without the need to create a full function block:

```
f = lambda x: x**2 + exp(x)
f(2.0) # Evaluates to 11.38905609893065
```

Lambda functions (1)

Consider the mathematical function $f(x) = x^2 + e^x$ defined as a Python function block:

```
def f(x):
    from math import exp
    return x**2 + exp(x)
```

Lambda functions provide a more compact way of defining simple functions, without the need to create a full function block:

```
f = lambda x: x**2 + exp(x)
f(2.0) # Evaluates to 11.38905609893065
```

This is often used when passing functions as arguments to other functions, e.g. in integration or optimization routines.

List comprehension basics

Sometimes, loops can be replaced by more compact constructs called *list comprehensions*.

```
L = [1,2,3,4,5]
for i in range(len(L)):
    L[i] = L[i]**2
```

List comprehension basics

Sometimes, loops can be replaced by more compact constructs called *list comprehensions*.

```
L = [1,2,3,4,5]
for i in range(len(L)):
   L[i] = L[i]**2
```

This can be replaced by:

```
L = [x**2 for x in L]
```

List comprehension basics

Sometimes, loops can be replaced by more compact constructs called *list comprehensions*.

```
L = [1,2,3,4,5]
for i in range(len(L)):
   L[i] = L[i]**2
```

This can be replaced by:

```
L = [x**2 for x in L]
```

List comprehensions can also include conditions:

```
L = [x**2 for x in L if x % 2 == 0]
```

Python classes

Sometimes, you may end up with many functions and data that are related to each other. In such cases, it is often useful to group them together into a single unit called a class.
A class is a blueprint for creating objects that share common properties and behaviors:

```
class Cat:
    def __init__(self, name, color):
        self.name = name
        self.color = color
        self.cuteness_level = 1

    def purr(self, cute_pts=10):
        print("Purr... =^.^= ")
        self.cuteness_level += cute_pts
        return

    def scratch(self, harshness=5):
        print("Scratch! >.< ")
        self.cuteness_level -= harshness
        return

    def get_cuteness(self):
        return self.cuteness_level
```

Python classes

Sometimes, you may end up with many functions and data that are related to each other. In such cases, it is often useful to group them together into a single unit called a class.
A class is a blueprint for creating objects that share common properties and behaviors:

```python
class Cat:
    def __init__(self, name, color):
        self.name = name
        self.color = color
        self.cuteness_level = 1

    def purr(self, cute_pts=10):
        print("Purr... =^.^= ")
        self.cuteness_level += cute_pts
        return

    def scratch(self, harshness=5):
        print("Scratch! >.< ")
        self.cuteness_level -= harshness
        return

    def get_cuteness(self):
        return self.cuteness_level
```

```python
my_cat = Cat("Arturo", "Gray")
my_cat.purr()
my_cat.scratch()
result = my_cat.get_cuteness()
print(f"Cuteness of {my_cat.name}: {result}")
```

```python
sweet_cat = Cat("Pinot", "White")
angry_cat = Cat("Rufus", "Black")

angry_cat.scratch(15)
sweet_cat.purr(25)

cats = [my_cat, sweet_cat, angry_cat]
cuteness_levels = [cat.get_cuteness() for cat in
      cats]
print(cuteness_levels)
```

Exercise: Classes and objects

Create a class `IdealGas` that represents an ideal gas using the ideal gas law. Store the name, number of moles and the gas constant as attributes. Implement a method `compute_pressure(V,T)` that computes the pressure given volume `V` and temperature `T`.

Exercise: Classes and objects

Create a class `IdealGas` that represents an ideal gas using the ideal gas law. Store the name, number of moles and the gas constant as attributes. Implement a method `compute_pressure(V,T)` that computes the pressure given volume `V` and temperature `T`.

```
class IdealGas:
    def __init__(self, name, n=1, R=8.314):
        """Create an ideal gas of n moles (using default R J/K/mol)"""
        self.name = name
        self.R = R
        self.n = n

    def compute_pressure(self,V,T):
        """Compute the pressure using P=nRT/V"""
        P = (self.n * self.R * T) / V
        return P

H2 = IdealGas("Hydrogen", 1.0)
N2 = IdealGas("Nitrogen", 2.5)
O2 = IdealGas("Oxygen", 1.333)

for gas in [H2,N2,O2]:
    pres = gas.compute_pressure(0.1,373)
    print(f'{gas.name} has a pressure of {pres:4.2f} Pa')
```

Today's outline

- Introduction
- Refresher
- New topics in programming
- Modules
- Conclusions
- Exercises

Using Modules in Python (1)

Modules are files containing Python code, used to organize functionalities and reuse code across projects. To use a module, it must first be imported using the `import` keyword. Here, we import the entire `math` module:

```
>>> import math
```

Using Modules in Python (1)

Modules are files containing Python code, used to organize functionalities and reuse code across projects. To use a module, it must first be imported using the `import` keyword. Here, we import the entire `math` module:

```
>>> import math
```

Once imported, use the dot notation to access functions and variables defined in the module:

```
>>> math.sqrt(16)
4.0
```

Using Modules in Python (1)

Modules are files containing Python code, used to organize functionalities and reuse code across projects. To use a module, it must first be imported using the `import` keyword. Here, we import the entire `math` module:

```
>>> import math
```

Once imported, use the dot notation to access functions and variables defined in the module:

```
>>> math.sqrt(16)
4.0
```

You can import specific attributes from a module using the `from ... import ...` syntax:

```
>>> from math import sqrt
>>> sqrt(16)
4.0
```

Using Modules in Python (2)

Alias module names using the `as` keyword to shorten module names and avoid naming conflicts:

```
>>> import numpy as np
```

Using Modules in Python (2)

Alias module names using the `as` keyword to shorten module names and avoid naming conflicts:

```
>>> import numpy as np
```

To view the list of all functions and variables in a module, use the `dir()` function:

```
>>> import math
>>> dir(math)
```

Using Modules in Python (2)

Alias module names using the `as` keyword to shorten module names and avoid naming conflicts:

```
>>> import numpy as np
```

To view the list of all functions and variables in a module, use the `dir()` function:

```
>>> import math
>>> dir(math)
```

Get help on how to use a module or a function using the `help()` function:

```
>>> help(math.sqrt)
```

The math module

Many mathematical operations and concepts are available in the math module:

```
from math import pi,sin,sqrt,log10\
,exp,floor,ceil,factorial,inf,log
print(pi)
print(sin(0.2*pi))
print(sqrt(2))
print(log10(10_000))
print(exp(1))
print(log(exp(2)))
print(floor(2.57))
print(ceil(2.57))
print(floor(-2.57))
print(round(2.4))
print(round(4.5))
print(factorial(5))
print(f"1 divided by infinity equals {1/inf}")
```

The math module

Many mathematical operations and concepts are available in the math module:

```
from math import pi,sin,sqrt,log10\
,exp,floor,ceil,factorial,inf,log
print(pi)
print(sin(0.2*pi))
print(sqrt(2))
print(log10(10_000))
print(exp(1))
print(log(exp(2)))
print(floor(2.57))
print(ceil(2.57))
print(floor(-2.57))
print(round(2.4))
print(round(4.5))
print(factorial(5))
print(f"1 divided by infinity equals {1/inf}")
```

```
3.141592653589793
0.5877852522924731
1.4142135623730951
4.0
2.718281828459045
2.0
2
3
-3
2
4
120
1 divided by infinity equals 0.0
```

Practice

Use the math module to compute $y = \sin(x)$ for 9 equidistant points $x \in [0, 2\pi]$, including end-points.

- Use *list comprehensions* to generate the lists `x` and `y`

Practice

Use the math module to compute $y = \sin(x)$ for 9 equidistant points $x \in [0, 2\pi]$, including end-points.

- Use *list comprehensions* to generate the lists `x` and `y`

```
1 from math import pi,sin
2 x_values = [x/8*2*pi for x in range(9)]
3 y_values = [sin(x) for x in x_values]
4
5 for x,y in zip(x_values,y_values):
6    print(f"{x: 10.4f},{y: 10.4f}")
```

Practice

Use the math module to compute $y = \sin(x)$ for 9 equidistant points $x \in [0, 2\pi]$, including end-points.

- Use *list comprehensions* to generate the lists `x` and `y`

```
from math import pi,sin
x_values = [x/8*2*pi for x in range(9)]
y_values = [sin(x) for x in x_values]

for x,y in zip(x_values,y_values):
   print(f"{x: 10.4f},{y: 10.4f}")
```

```
0.0000, 0.0000
0.7854, 0.7071
1.5708, 1.0000
2.3562, 0.7071
3.1416, 0.0000
3.9270, -0.7071
4.7124, -1.0000
5.4978, -0.7071
6.2832, -0.0000
```

Today's outline

- Introduction
- Refresher
- New topics in programming
- Modules
- Conclusions
- Exercises

In conclusion...

- Python: A versatile development language. Easy to use libraries makes this language multi-purpose and easy to use.
- Programming basics: variables, operators and functions, locality of variables, modules and recursive operations

In conclusion...

- Python: A versatile development language. Easy to use libraries makes this language multi-purpose and easy to use.
- Programming basics: variables, operators and functions, locality of variables, modules and recursive operations
- For now: exercises on slide deck and Python modules

In conclusion...

- Python: A versatile development language. Easy to use libraries makes this language multi-purpose and easy to use.
- Programming basics: variables, operators and functions, locality of variables, modules and recursive operations
- For now: exercises on slide deck and Python modules

Practice vectors and arrays

1. Create a list `x` with the elements:
   - `[2, 4, 6, 8, ..., 16]`
   - `[0, 0.5, 2/3, 3/4, ..., 99/100]`
2. Create a list `x` with the elements: $x_n = \frac{(-1)^n}{2n-1}$ for $n = 1, 2, 3, \ldots, 200$. Find the sum of the first 50 elements $x_1, \ldots, x_{50}$.
3. Let `x = list(range(20, 201, 10))`. Create a list `y` of the same length as `x` such that:
   - `y[i] = x[i] - 3`
   - `y[i] = x[i]` for every even index *i* and `y[i] = x[i] + 11` for every odd index *i*.
4. Let `T = np.array([[3, 4, 6], [1, 8, 6], [-4, 3, 6], [5, 6, 6]]).T`. Perform the following operations on T:
   - Retrieve a list consisting of the 2nd and 4th elements of the 3rd row.
   - Find the minimum of the 3rd column.
   - Find the maximum of the 2nd row.
   - Compute the sum of the 2nd column
   - Compute the mean of the row 1 and the mean of row 3

Practice plotting

1. Plot the functions $f(x) = x$, $g(x) = x^3$, $h(x) = e^x$ and $z(x) = e^{x^2}$ over the interval $[0, 4]$ on the normal scale and on the log-log scale. Use an appropriate sampling to get smooth curves. Describe your plots by using the functions: `plt.xlabel`, `plt.ylabel`, `plt.title` and `plt.legend`.

2. Make a plot of the functions: $f(x) = sin(1/x)$ and $g(x) = cos(1/x)$ over the interval $[0.01, 0.1]$. How do you create `x` so that the plots look sufficiently smooth?

Practice control flow and loops (1)

1. Write a function that uses two logical input arguments with the following behaviour:

$$f(\text{true}, \text{true}) \mapsto \text{false}$$
$$f(\text{false}, \text{true}) \mapsto \text{true}$$
$$f(\text{true}, \text{false}) \mapsto \text{true}$$
$$f(\text{false}, \text{false}) \mapsto \text{false}$$

2. Write a function that computes the factorial of x:

$$f(x) = x! = 1 \times 2 \times 3 \times 4 \times \ldots \times x$$

- Using a loop-construction
- Using recursion

Practice control flow and loops (2)

1. Write a function that computes the exponential function using the Taylor series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

   until the last term is smaller than 10^{-6}.

2. Use a script to compute the result of the following series:

$$f_n = \sum_{n=1}^{\infty} \frac{1}{\pi^2 n^2}$$

   This should give you an indication of the fraction this series converges to.

   - Now plot in two vertically aligned subplots i) The result as a function of n, and ii) the difference with the earlier mentioned fraction as a function of n. For the latter, consider carefully the axis scale!

Practice logical indexing

1. Let `x = np.linspace(-4,4,1000)`, $y_1 = 3x^2 - 4x - 6$ and $y_2 = 1.5x - 1$. Use logical indexing to determine function $y_3 = \max(\max(y_1, y_2), 0)$. Plot the function.
2. Consider these data concerning the age (in years), length (in cm) and weight (in kg) of twelve adult men: `A = [41 25 33 29 64 34 47 38 49 32 26 26]; H = [165 186 177 190 156 174 164 205 184 190 165 171]; W = [75 90 97 60 74 65 101 85 91 75 87 70];`.
   - Calculate the average of all vectors (age, weight and length).
   - Combine the command `length` with logical indexing to determine how many men in the group are taller than 182 cm.
   - What is the average age of men with a body-mass index ($B \equiv \frac{W}{L^2}$ with W in kg and L in m) larger than 25? And for men with a $B < 25$?
   - How many men are older than the average and at the same time have a BMI below 25?

Practice algorithm: Fourier series for heat equation

The unsteady 1D heat equation in 1D in a slab of material is given as:

$$\frac{\partial T}{\partial t} = k\frac{\partial^2 T}{\partial x^2}$$

We can express the temperature profile $T(x,t)$ in the slab using a Fourier sine series. For an initial profile T(x,0) = 20 and fixed boundary values T(0,t) = T(L,t) = 0, the solution is given as:

$$T(x,t) = \sum_{n=1}^{n=\infty} \frac{40(1-(-1)^n)}{n\pi} \sin\left(\frac{n\pi x}{L}\right) \exp\left(-kt\frac{n\pi^2}{L}\right)$$

- Create a script to solve this equation using loops and/or conditional statements